



FTP



- The File Transfer Protocol (FTP) has been around since 1971
- Remarkably, the underlying protocol itself has undergone little change since then
- Clients and servers, on the other hand, have been almost constantly improved and refined
- The **vsftpd** program is a popular FTP server implementation and is used by major high-traffic FTP sites

Mechanics of FTP



- Although this information isn't crucial to your being able to get an FTP server up and running, it is important when you need to troubleshoot FTP issues—especially issues that don't clearly manifest themselves as being related to FTP
- These details might help you pinpoint issues such as whether the problem is with the network, FTP server, or FTP client
- **Client/Server Interactions**
 - FTP facilitates the exchange of files between an FTP client and an FTP server, its design has some built-in nuances
 - One of its nuances stems from the fact that it uses two ports: a **control port (port 21)** and a **data port (port 20)**
 - The control port serves as a communication channel between the client and the server for the exchange of commands and replies
 - The data port is used purely for the exchange of data, which can be a file, part of a file, or a directory listing
 - FTP can operate in two modes: **active** FTP mode and **passive** FTP mode



- Active FTP was traditionally used in the original FTP specifications
- In this mode, the client connects from an ephemeral port (a port number greater than 1024) to the FTP server's command port (port 21)
- When the client is ready to transfer data, the server opens a connection from its data port (port 20) to the IP address and ephemeral port combination provided by the client
- The key here is that the client does not make the actual data connection to the server but instead informs the server of its own port by issuing the PORT command; the server then connects back to the specified port
- The server can be regarded as the active party (or the agitator) in this FTP mode
- From the perspective of an FTP client that is behind a firewall, the active FTP mode poses a slight problem: the firewall on the client side might frown upon (or disallow) connections originating or initiated from the Internet from a privileged service port (such as data port 20) to non-privileged service ports on the clients it is supposed to protect

Passive FTP



- The FTP client issues the **PASV** command to indicate that it wants to access data in passive mode, and the server then responds with an IP address and an ephemeral port number on itself to which the client can connect to transfer the data
- The PASV command issued by the client tells the server to “listen” on a data port that is not its normal data port (that is, port 20) and to wait for a connection rather than initiate one
- The key difference in comparison with active FTP mode is that in passive FTP mode, it is the client that initiates the connection to the port and IP address provided by the server
- In this regard, the server can be considered the passive party in the data communication
- From the perspective of an FTP server that is behind a firewall, passive FTP mode is a little problematic because a firewall’s natural instinct would be to disallow connections that originate from the Internet that are destined for ephemeral ports of the systems that it is supposed to protect
- A typical symptom of this behavior occurs when a client appears to be able to connect to the server without a problem, but the connection seems to hang whenever an attempt to transfer data occurs
- To address some of the issues pertaining to FTP and firewalls, many firewalls implement application-level proxies for FTP that keep track of FTP requests and open up those high ports when needed to receive data from a remote site

Configuration



■ Installation

- `sudo yum install vsftpd ftp`

■ User creation

- `sudo useradd user1`
- `sudo passwd user1`
- `sudo vim /etc/vsftpd/chroot_list`

■ Configuration

- `sudo vim /etc/vsftpd/vsftpd.conf`
- `pam_service_name=vsftpd`
- `userlist_enable=YES`
- `use_localtime=YES`
- `chroot_local_user=YES`
- `chroot_list_enable=YES`

■ Start Service

- `sudo systemctl enable vsftpd`
- `sudo systemctl start vsftpd`

■ Add Firewall Rule

- `sudo firewall-cmd --add-service=ftp --permanent`
- `sudo firewall-cmd --reload`



NFS



- Network File System (NFS) is one of the native ways of sharing files and applications across the network in the Linux world
- NFS is somewhat similar to Microsoft Windows File Sharing, in that it allows you to attach to a remote file system (or disk) and work with it as if it were a local drive—a handy tool for sharing files and large storage space among users
- NFS and Windows File Sharing are solutions to the same problem; however, the solutions are very different beasts
- NFS requires different configurations, management strategies, tools, and underlying protocols

Mechanics of NFS



- As with most network-based services, NFS follows the usual client and server paradigms—that is, it has its client-side components and its server-side components
- For clientA to access the /home share being exported by serverA, clientA needs to make an NFS mount request for /home so that it can mount it locally, such that the remote share appears locally as the /home directory
- Here is a simple command to trigger this
 - `[master@clientA ~]$ sudo mount serverA:/home /home`
- After executing the previous command on clientA, all the users on clientA would be able to view the contents of /home as if it were just another directory or local file system
- Linux would take care of making all of the network requests to the server
- **Remote procedure calls (RPCs)** are responsible for handling the requests between the client and the server
- RPC technology provides a standard mechanism for any RPC client to contact the server and find out to which service the calls should be directed
- Thus, whenever a service wants to make itself available on a server, it needs to register itself with the RPC service manager, **portmap**, which tells the client where the actual service is located on the server

NFS Versions



- The protocol behind NFS has evolved and changed a lot over the years
- **NFSv2**
 - Mount requests are granted on a per-host basis and not on a per-user basis
 - This version uses Transmission Control Protocol (TCP) or User Datagram Protocol (UDP) as its transport protocol
 - Version 2 clients have a file size limitation of less than 2GB that they can access
- **NFSv3**
 - This version includes a lot of fixes for the bugs in NFSv2
 - It has more features than version 2, has performance gains over version 2, and can use either TCP or UDP as its transport protocol
 - Depending on the local file system limits of the NFS server itself, clients can access files larger than 2GB in size
 - Mount requests are also granted on a per-host basis and not on a per-user basis
- **NFSv4**
 - This version of the protocol uses a stateful protocol such as TCP or Stream Control Transmission Protocol (SCTP) as its transport
 - It has improved security features thanks to its support for Kerberos;
 - It was designed with the Internet in mind. This version of the protocol is firewall-friendly and listens on the well-known port 2049
 - The services of the RPC binding protocols (such as `rpc.mountd`, `rpc.lockd`, and `rpc.statd`) are no longer required in this version of NFS because their functionality has been built into the server; in other words, NFSv4 combines these previously disparate NFS protocols into a single protocol specification. (The portmap service is no longer necessary.)
 - It includes support for file access control list (ACL) attributes and can support both version 2 and version 3 clients

Configuration on Server



- Install nfs-server
 - `sudo yum -y install nfs-utils`
- Update configuration
 - `sudo vim /etc/idmapd.conf`
- Add share directories
 - `sudo vim /etc/exports`
 - `sudo exportfs -a`
- Start the servers
 - `sudo systemctl start nfs-server`
 - `sudo systemctl start rpcbind`
- Add firewall service
 - `sudo firewall-cmd --add-service=nfs3 --permanent`
 - `sudo firewall-cmd --add-service=mountd --permanent`
 - `Sudo firewall-cmd --add-service=rpc-bind --permanent`

Configuration on Client



- Install nfs-server
 - `sudo yum -y install nfs-utils`
- Verify the NFS share
 - `sudo showmount -e <server ip>`
- Mount the nfs volume
 - `sudo mount -t nfs <server ip>:<nfs share> <mount point>`



Samba



- Samba is a powerful suite of applications that provides an open source implementation of the Server Message Block/ Common Internet File System (**SMB/CIFS**) protocol
- Among other things, Samba helps Linux-based systems interoperate with Windows-based operating systems
- Samba transparently provides file and print sharing services to Windows clients as well as other networked clients running other operating systems
- It does this through the use of the native Microsoft networking protocols **SMB/CIFS**
- From a system administrator's point of view, this means you can deploy a Linux-based server and use it to provide file-sharing, authentication, print, and other services to other non-native Linux clients such as Microsoft Windows systems
- Using Samba means that Windows systems can use their native tongue to talk to the Linux server—which means fewer hassles for you and seamless integration for your users

Mechanics of Samba



- Relative to Samba, several options are available for handling username and password issues in heterogeneous environments, including the following:
 - **Linux pluggable authentication modules (PAMs)**
 - Allow you to authenticate users against a DC
 - This means you still have two user lists—one local and one on the DC—but your users need to keep track of their passwords only on the Windows system
 - **Samba as a DC**
 - Allows you to keep all your logins and passwords on the Linux system, while all your Windows boxes authenticate with Samba
 - When Samba is used with a Lightweight Directory Access Protocol (LDAP) back-end for this, you will have a scalable and extensible solution
 - **Custom script**
 - Allows you to use your own custom script
 - For sites with a well-established system for maintaining logins and passwords, it isn't unreasonable to come up with a custom script. This can be done using a scripting language with good cross-platform support.
 - Such scripts can be coaxed to allow changes to the Security Access Manager (SAM) to update the DC's password list

Configuration



- Install server

- `sudo yum install samba`

- Configure share

- `sudo vim /etc/samba/smb.conf`

[share]

`comment = My Share`

`path = /var/smb/share`

`writable = yes`

`public = yes`

`browseable = yes`

`create mask = 0644`

`directory mask = 0755`

`write list = user`

- Add samba user

- `sudo smbpasswd -a <username>`

- Start server

- `sudo systemctl enable smb`
- `sudo systemctl start smb`

- Add firewall rules

- `sudo firewall-cmd --add-service=samba --permanent`
- `sudo firewall-cmd --reload`